In Figure 28, we report the ICL n-shot performance of each of these two tasks for GPT-J, and each model in the Llama 2 family. We find that the models perform this task well given 10 example pairs, with the performance of the next-item task being higher than the previous-item task. Performance generally increases when more examples are provided.

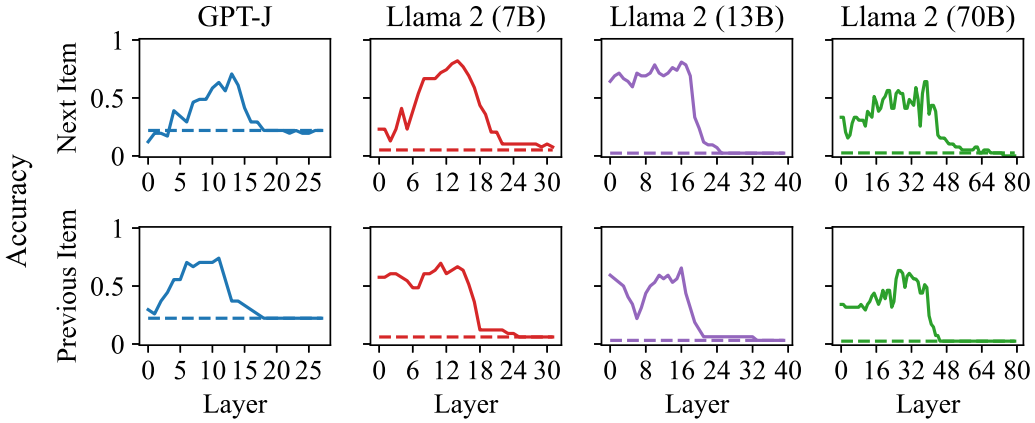

Figure 29: Zero-shot accuracy on the "Next Item" and "Previous Item" cylic tasks for GPT-J and all sizes of Llama 2. We show model accuracies before adding the function vector (dotted lines) and after adding the FV to a specific layer (solid lines). The function vector improves performance significantly for both tasks compared to the model's zero-shot baseline.

We extract a function vector for each of these tasks and evaluate their performance in the zero-shot setting, adding the function vector to different layers of the network. In Figure 29, we report the zero-shot accuracy of each model before adding the function vector with a dashed line, and the accuracy after adding a function vector with a solid line. We see that the function vector significantly improves performance for each task compared to the zero-shot baseline. In addition, the trends for these datasets generally follows those previously reported for other tasks (cyclic or not). The peak performance is achieved when adding the FV to early-middle layers, and there is a sharp drop in performance about $2/3$ of the way through the network.

Table 22: A few example outputs of adding the "next-item" and "previous-item" function vectors to layer 9 of GPT-J. We see that the function vectors are able to correctly trigger the cyclic behavior of "next" or "previous" when presented with a boundary case, while the base model usually defaults to copying the input query.

Input Prompt:	'Q: Sunday\nA:'	'Q: December\nA:'	'Q: z\nA:'	'Q: seven\nA:'	'Q: 21\nA:'	'Q: Monday\nA:'	'Q: January\nA:'
GPT-J	Sunday	December	z	eight	I don't know	Tuesday	January
GPT-J+Next-Item FV	Monday	January	a	eight	22	Tuesday	February
GPT-J+Previous-Item FV	Saturday	November	y	six	20	Sunday	December

In Table 22, we include a few example outputs of zero-shot prompts for both the baseline model and the model when we add the corresponding function vectors of either "next-item" or "previous-item", showing their ability to correctly induce the "next" or "previous" cyclic behavior.

For antonyms, the cyclic behavior gives a contradiction in two additions. That is, given a word w_1 and a vector offset v that can give the antonym of w_1, then we expect to return to w_1 after adding v again (i.e $w_1 + 2 * v = w_1$). The cyclic subsets in studied here have longer cycles (e.g. for days of the week, and an offset v', we'd expect $w_2 + 7 * v' = w_2$), but the same reasoning applies. Because FVs can trigger the corresponding cyclic behavior, this provides additional evidence that they are not just doing simple word vector arithmetic.

M AN ALTERNATIVE EVALUATION OF FUNCTION VECTORS

Recall that we define a function vector (v_t) for a particular task (t) as the sum of the task-conditioned mean activations ($\bar{a}^t_{\ell j}$) over a small set of attention heads ($\mathcal{A}$) (see equation 5, Section 2.3). Given a function vector created in this manner, we can test its causal effects by adding it to a single hidden state of a model at a particular layer and measuring its ability to trigger a particular task behavior (see Appendix B.2 for more details).

An alternative approach to test whether the outputs of the attention heads in $\mathcal{A}$ can trigger a particular task behavior is to instead add their task-conditioned mean activations ($\bar{a}^t_{\ell j}$) to their corresponding layer's hidden states, and to do so at every layer that is implicated by the heads contained in $\mathcal{A}$. This is in contrast to the FV, which adds all these attention head outputs to a single layer.

As the model performs computation at layer k, the alternative approach updates the hidden state $\mathbf{h}_k$ by adding the task-conditioned mean activations of all heads in $\mathcal{A}$ that output to layer k. If we represent the attention heads in $\mathcal{A}$ with (layer, head index) tuples, then we write the updated hidden state $\mathbf{h}_k^{'}$ as:

$$\mathbf{h}_k^{'} = \mathbf{h}_k + \sum_{(\ell,j)\in\mathcal{A} \mid \ell=k} \bar{a}^t_{\ell j} \tag{16}$$

We perform the update intervention as specified in equation 16 for all layers represented by $\mathcal{A}$.

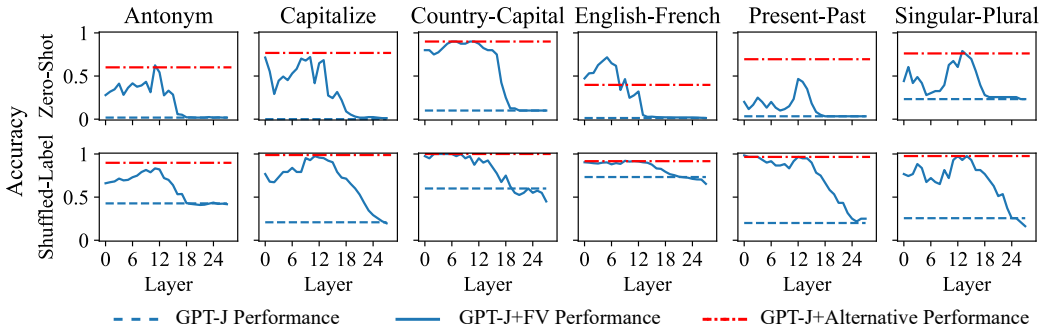

Figure 30: Comparing the causal effects of function vectors (solid blue line) and an alternative approach (red dashed line), which adds the components of an FV to their respective layers instead of at a single concentrated layer. The model baseline in each setting is shown with a dashed blue line. Zero-shot results for 6 tasks are shown in the first row, and shuffled-label results are shown in the second row. In the zero-shot setting, the alternative approach matches FV performance for most tasks. It performs worse for English-French translation and better on the Present-Past task. In the shuffled-label setting, the alternate approach matches FV performance for all tasks.

In Figure 30, we compare the causal effects of the alternative approach described in equation 16 to the original function vector formulation for both zero-shot and shuffled-label contexts across our 6 representative tasks using GPT-J. The base model performance is shown with a dashed blue line, and the solid blue line shows performance when we add the function vector to layer ℓ. The results of the alternative approach are shown with a red dashed line.

In the zero-shot setting, the alternative approach matches the performance of the function vector for a majority of the tasks. It performs worse on English-French, and better on the Present-Past task. In the shuffled-label setting, the alternative approach matches the causal effects of function vector for all tasks.

On average, we find that using the alternative approach to measure the causal effects of $\mathcal{A}$ compared to the original function vector formulation works about as well, typically achieving the same peak performance. However, the function vector approach does highlight an interesting phenomena of performance dropoff around 2/3 of the way through the network which is not possible to see when using the alternative approach.

N INVESTIGATING FUNCTION VECTOR EFFECTS IN VOCABULARY SPACE

In the main paper (Section 3.2, Table 6), we examine quantitative evidence that the action of a function vector (FV) cannot be explained by simply boosting a set of words directly encoded by the function vector in the style of Dar et al. (2023), see Appendix A for a more detailed discussion. In this section we examine the causal effects of several function vectors in vocabulary space to understand the relationship between the words that are encoded in a function vector and the words that are boosted by the transformer when we intervene with an FV.

Unlike previous analysis, in this section we investigate how adding a function vector (v_t) to layer ℓ changes the distribution of log probabilities over a set of relevant tokens ($w_i \in W$) compared to the baseline model's response. That is, for a token w_i we compute:

$$\triangle\text{logprob} = \log(f(p^t \mid \mathbf{h}_\ell := \mathbf{h}_\ell + v_t)[w_i]) - \log(f(p^t)[w_i]) \tag{17}$$

We investigate the tokens with the highest increase in log probabilities under FV intervention and include a few examples of the behavior we observe in Table 23. Here we show a few examples of the tokens with the largest $\triangle$logprob for three tasks: Country-Capital, Antonym, English-French.

Table 23: The tokens with the highest increase in $\triangle$logprob for different queries on three tasks - Country-Capital, Antonym, and English-French (shown in black text). For comparison, we present the $\triangle$logprob of the top tokens we get when decoding the FV via $D(v_t)$ (shown in red text directly below). The $\triangle$logprob of the $D(v_t)$-tokens is much lower than the query-specific answers. In general, the tokens promoted the most correspond to likely answers to the specific query, rather than generic tokens in the output space. In the case of '`wolf`' for the English-French task, the model answers incorrectly. However, examining $\triangle$logprob indicates several likely answers are still promoted – showing FVs have causal effects that are not adequately captured using top-1 accuracy.

	Tokens with largest positive $\triangle$logprob under FV intervention
Country-Capital	
South Africa	' Pret' (+4.7), ' Johannes' (+4.2), ' Dur' (+4.0), ' Cape' (+3.9), ' Kimber' (+3.7)
	' London' (+1.3), ' Moscow' (+1.1), ' Paris', (+1.0), ' Bangkok', (+0.3) ' Madrid' (+0.2)
Syria	' Damascus' (+4.9), ' Tart' (+4.2), ' Raqqa' (+4.1), ' Dam' (+4.0), ' Aleppo' (+3.8)
	' London' (+2.2), ' Moscow' (+2.1), ' Paris', (+2.1), ' Bangkok', (+1.3) ' Madrid' (+2.1)
Antonym	
temporary	' perpetual' (+4.4), ' definitive' (+4.3),' everlasting' (+4.1), ' permanent' (+3.7)
	' counterpart' (+1.3), ' lesser' (+0.9), ' destroy' (+0.5), ' negate' (+0.4), ' wrong' (-0.8)
static	' evolving' (+4.0), ' flexible' (+3.8), ' polymorph' (+3.7), ' dynamic' (+3.7)
	' counterpart' (+0.4), ' lesser' (-0.3), ' destroy' (0.2), ' negate' (-0.4), ' wrong' (-1.7)
English-French	
wolf	' lou' (+6.3), ' ours' (+6.2), ' chau' (+5.8), ' dé' (+5.8), ' Lou' (+5.7)
	' âĶĪ' (-1.4), ' masc' (-0.6), ' ç¥l' (-0.9), ' embr' (+2.4), ' è' (+1.6)
advertisement	' ann' (+7.6), ' aff' (+6.8), 'annon' (+6.7), ' ré' (+6.2), ' pub' (+6.14)
	' âĶĪ' (0.2), ' masc' (1.3), ' ç¥l' (-0.2), ' embr' (-0.5), ' è' (+0.6)

For the country-capital task, the tokens with the highest increase in log probability typically correspond with likely answers to the specific query, rather than answers to the task in general. For example, given the query '`South Africa`', the country-capital FV promotes '`Pretoria`', in addition to other cities in South Africa. We compare the 5 tokens with the highest overall increase in log probability, and the top 5 tokens we get from $D(v_t)$, which are shown below these in red. We see a substantial difference between the magnitudes of the $\triangle$logprob for these tokens and the tokens that were promoted the most.

We see a similar trend for Antonyms - where the promoted tokens are all reasonably valid antonyms of the query word, rather than just antonyms in general.

For English-French, the query ' `wolf`' is not answered correctly under intervention, but the correct translation (' `loup`') is still promoted by the FV when we examine the $\triangle$logprob. Similarly, for the query '`advertisement`', the dataset target is '`publicité`', but prefix tokens for another valid french translation (' `annonce`') are also promoted when examining the $\triangle$logprob distribution - ' `ann`', and '`annon`'.